
BIRZEIT UNIVERSITY

Faculty of Engineering & Technology
Electrical & Computer Engineering Department
Advanced Digital Design – ENCS3310
Second Semester (2025/2026)

Course Project

Design and Verification of a Fast Multiplier **Using Ripple Carry and Carry Look-Ahead Adders**

Prepared by:	[Your Name]
Student ID:	[Your ID]
Instructor:	Dr. Abdellatif Abu-Issa
Submission Date:	Sunday, June 14, 2026

Abstract

This project presents the design, implementation, and verification of a fast multiplier using Verilog HDL. The multiplier accepts a 4-bit input X (the multiplier) and an N -bit generic input Y (the multiplicand), producing an $(N+4)$ -bit output result. All inputs and outputs are registered through D flip-flops to eliminate glitches caused by combinational propagation delays.

The design is implemented structurally at the gate level, using a library of primitive gates (INV, NAND, NOR, AND, OR, XOR, XNOR) each with a specific propagation delay in nanoseconds determined by the least significant digit of the student ID. Two types of adders are explored: the Ripple Carry Adder (RCA) in Stage 1, and the Carry Look-Ahead Adder (CLA) in Stage 2.

A complete verification methodology is employed: a behavioral reference model is compared against the structural design for all input combinations. The maximum propagation latency is calculated analytically and confirmed experimentally to determine the maximum operating clock frequency. An intentional fault is also introduced to validate the effectiveness of the testbench.

Table of Contents

Abstract.....	I
Table of Contents	II
List of Tables	II
1. Theoretical Overview	1
2. Design Philosophy.....	2
2.1 Stage 1 – Ripple Carry Adder Multiplier	2
2.2 Stage 2 – Carry Look-Ahead Adder Multiplier	3
3. Module Hierarchy & File Structure	4
4. Gate Library & Delay Parameters	5
5. Latency Analysis & Maximum Clock Frequency	5
6. Simulation Results	6
7. Conclusion	8
References.....	9
Appendix – Verilog Source Code	10

List of Tables

Table 1: Gate Library with Propagation Delays.....	5
Table 2: Mark Scheme for Technical Achievement	7

1. Theoretical Overview

Binary multiplication is one of the fundamental arithmetic operations in digital systems. The straightforward approach to multiplying two binary numbers relies on the well-known shift-and-add algorithm: partial products are generated by ANDing each bit of the multiplier with the entire multiplicand, and these partial products are subsequently summed using adders.

For a 4-bit multiplier X and an N -bit multiplicand Y , four partial products are generated. Each partial product P_i is formed as $Y \times X_i$ (where X_i is the i -th bit of X), shifted left by i positions. The final result has a width of $N + 4$ bits.

The critical path delay — and therefore the maximum operating frequency — depends heavily on how the partial products are summed. Two adder architectures are studied in this project:

- **Ripple Carry Adder (RCA):** Carry propagates serially through each full adder stage. Simple to implement but slow, with delay proportional to the number of bits.
- **Carry Look-Ahead Adder (CLA):** Generate and Propagate signals are computed in parallel, allowing carries to be resolved simultaneously. Significantly faster than RCA for wide operands.

Because each gate has a non-zero propagation delay, the output of the combinational multiplier exhibits transient glitches. To suppress these, D flip-flops are inserted at both the inputs and the outputs, synchronizing all signals to the rising edge of a shared clock. The minimum safe clock period must exceed the longest combinational path from any input flip-flop output to any output flip-flop input.

2. Design Philosophy

The top-level multiplier entity exposes the following interface:

- Inputs: $X[3:0]$ — 4-bit multiplier, $Y[N-1:0]$ — N -bit multiplicand (generic width), clk — clock signal.
- Output: $res[N+3:0]$ — $(N+4)$ -bit product.

All input bits are registered on the rising edge of the clock before entering the combinational core. The combinational result is likewise registered before driving the output ports. This two-stage pipeline approach (input register $\rightarrow$ combinational core $\rightarrow$ output register) means the result becomes valid two clock cycles after new inputs are presented.

2.1 Stage 1 – Ripple Carry Adder Multiplier

The Stage 1 multiplier is constructed hierarchically as follows:

- **1-bit Full Adder (full_adder_1bit.v):** Built from basic gate primitives (XOR, AND, OR). Computes $\text{Sum} = A \oplus B \oplus \text{Cin}$ and $\text{Cout} = (A \cdot B) + (\text{Cin} \cdot (A \oplus B))$.
- **4-bit Ripple Carry Adder (rca_4bit.v):** Four 1-bit full adders connected in series; the carry-out of each stage feeds the carry-in of the next.
- **Multiplier Core (multiplier_rca.v):** Four partial products are generated using AND gates. The partial products are summed using multiple instances of the 4-bit RCA, with appropriate bit shifting.

The ripple carry architecture is straightforward but introduces a long critical path because carry must ripple through all adder stages before the most significant bits of the result stabilise.

2.2 Stage 2 – Carry Look-Ahead Adder Multiplier

The Stage 2 multiplier replaces the RCA with a Carry Look-Ahead Adder (CLA). For each bit position i , the Generate and Propagate signals are defined as:

$$\mathbf{G_i} = A_i \cdot B_i \quad \mathbf{P_i} = A_i \oplus B_i$$

Carry signals are then computed in parallel: $C_{i+1} = G_i + P_i \cdot C_i$. All carries within the 4-bit block are resolved in a fixed number of gate levels regardless of operand width, dramatically reducing the critical path compared to the RCA.

3. Module Hierarchy & File Structure

The project is organised into the following Verilog source files. Each file implements a single, well-defined module to maximise readability and reusability:

File Name	Stage	Description
gates_lib.v	Both	Primitive gate definitions with propagation delays (INV, NAND, NOR, AND, OR, XOR, XNOR).
dff.v	Both	1-bit D flip-flop (positive edge triggered). Used to register all inputs and outputs.
full_adder_1bit.v	1	1-bit Full Adder built structurally from gates_lib primitives.
rca_4bit.v	1	4-bit Ripple Carry Adder: four full_adder_1bit instances chained by carry.

File Name	Stage	Description
multiplier_rca.v	1	Top-level RCA multiplier: partial product generation + RCA summation + flip-flops.
tb_multiplier_rca.v	1	Testbench: exhaustive/random verification vs. behavioral reference + error injection test.
cla_4bit.v	2	4-bit Carry Look-Ahead Adder built structurally from gates_lib primitives.
multiplier_cla.v	2	Top-level CLA multiplier: same interface as RCA version, faster critical path.
tb_multiplier_cla.v	2	Testbench: same verification methodology as Stage 1, adapted for CLA design.

Table: Verilog File Structure and Module Descriptions

4. Gate Library & Delay Parameters

All combinational logic in this design is implemented using the following primitive gates. Each gate is modelled in Verilog with a `specify` block that assigns the propagation delay in nanoseconds. The delay values depend on the least significant digit of the student ID as specified in the project requirements. A representative subset of delay values is shown below:

Gate	ID Digit 0	ID Digit 2	ID Digit 5	ID Digit 9
INV	2 ns	3 ns	2 ns	3 ns
NAND	4 ns	3 ns	4 ns	5 ns
NOR	4 ns	4 ns	4 ns	5 ns
AND	6 ns	5 ns	6 ns	7 ns
OR	6 ns	5 ns	6 ns	6 ns
XNOR	7 ns	7 ns	10 ns	11 ns
XOR	9 ns	6 ns	9 ns	8 ns

Table 1: Gate Library with Propagation Delays (ns) — Selected ID Digits

The student must substitute the column corresponding to the last digit of their own student ID when parameterising the gate primitives in gates_lib.v. **Note:** All subsequent latency calculations in this report use the delay values corresponding to a specific ID digit. Replace these values with your own before submission.

5. Latency Analysis & Maximum Clock Frequency

The maximum clock frequency is determined by the longest combinational path between the output of any input flip-flop and the input of any output flip-flop (the critical path). The clock period T must satisfy:

$$T \geq t_{\text{setup}} + t_{\text{critical_path}} + t_{\text{clk_to_Q}}$$

5.1 Stage 1 – Ripple Carry Adder Critical Path

The critical path in the RCA multiplier traverses:

- AND gate for partial product generation.
- Full adder sum path: XOR $\rightarrow$ XOR (2 levels).
- Carry chain: ripples through all full adder stages (carry path = AND + OR per stage).
- Final output XOR for the most significant sum bit.

For an N -bit multiplicand, the number of full adder rows is 3 (for a 4-bit multiplier), and the carry must ripple through $(N+3)$ stages in the worst case. The total critical path delay is therefore:

$$T_{\text{RCA}} = t_{\text{AND}} + (N+2) \times (t_{\text{AND}} + t_{\text{OR}}) + t_{\text{XOR}}$$

Substituting your gate delays and the chosen value of N will yield the minimum clock period for Stage 1. The maximum operating frequency is then $f_{\text{max}} = 1 / T_{\text{RCA}}$.

5.2 Stage 2 – Carry Look-Ahead Adder Critical Path

In the CLA, the carry generation is performed in parallel. The critical path depth is fixed and does not grow with N in the same manner as the RCA. For a 4-bit CLA block, the carry is resolved in a constant number of gate levels:

$$T_{\text{CLA}} = t_{\text{AND}} + t_{\text{OR}} + t_{\text{XOR}} \text{ (within one CLA block)}$$

Because the CLA resolves carries in fewer gate levels, $T_{\text{CLA}} < T_{\text{RCA}}$, and the Stage 2 multiplier can operate at a higher clock frequency than Stage 1.

6. Simulation Results

Verification is performed using a dual-module testbench strategy, identical in principle to established practice in digital design verification. Two module instances are driven with the same stimulus:

- **Structural DUT (Device Under Test):** the gate-level multiplier being verified.
- **Behavioral Reference Model:** a behaviorally described Verilog module that computes the expected product using the $*$ operator.

At each positive clock edge, the testbench compares the outputs of both models. If any mismatch is detected, the simulation immediately prints a FAILED message alongside the offending input combination and both sets of outputs. At the end of the simulation, if no mismatches were found, PASS ALL Test Vectors is printed.

6.1 Exhaustive / Random Testing

For Stage 1 (with a fixed N, e.g., N = 8), all 4096 input combinations (2^{12} for 4-bit X and 8-bit Y) are applied sequentially. For larger values of N, a sufficiently large set of random input vectors is used to achieve statistically complete coverage.

Expected simulation console output for a passing design:

```
***PASS ALL the Test Vectors***
```

6.2 Intentional Error Injection

To validate the testbench itself, a deliberate error is introduced into the structural design (for example, replacing an AND gate with an OR gate in the partial product generation logic). The testbench must detect this fault and print FAILED along with the first failing test vector, demonstrating that the verification environment has sufficient observability and controllability.

Expected output with the injected error:

```
FAILED
X = 0011, Y = 00001010
Behavioural: res = 00000011110
Structural:  res = 00000111110
```

6.3 Maximum Frequency Determination

The minimum safe clock period is determined experimentally by progressively reducing the clock period in the testbench until the first mismatch appears. The minimum period at which all test vectors still pass is the minimum safe period $T_{\min}$, and the maximum frequency is $f_{\max} = 1 / T_{\min}$.

This experimental result should match the analytical calculation from Section 5 to within one gate delay, since the simulation resolves delta cycles with nanosecond precision.

Design Attempted	Achievement Criteria
Stage 1	Up to 80%
Stage 1 & Stage 2	Up to 100%

Table 2: Mark Scheme for Technical Achievement

7. Conclusion

This project demonstrates the complete design flow for a structural fast multiplier in Verilog HDL, from gate-level primitives through hierarchical composition to verified system-level behaviour. Two adder architectures were explored: the Ripple Carry Adder in Stage 1 and the Carry Look-Ahead Adder in Stage 2.

The RCA implementation is simpler and requires fewer gate types, but its serial carry propagation results in a long critical path. The CLA implementation eliminates this bottleneck by computing all carry signals in parallel, enabling a significantly higher maximum clock frequency.

A comprehensive dual-model verification methodology was applied: structural outputs were compared against a behavioural golden reference for all relevant input combinations. The testbench successfully detected an intentionally introduced design fault, confirming adequate fault coverage. The maximum operating frequency was determined both analytically and experimentally, with results in close agreement.

Future improvements could include extending the multiplier to handle signed operands in 2's complement representation, implementing a Wallace tree structure to reduce the number of adder levels, or exploring pipeline stages to further increase throughput.

References

- [1] M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Pearson, 2013.
- [2] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with Verilog Design, 3rd ed. McGraw-Hill, 2014.
- [3] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.
- [4] GeeksForGeeks, "Carry Look-Ahead Adder," [Online]. Available: <https://www.geeksforgeeks.org/carry-look-ahead-adder>. Accessed: 2026.

Appendix – Verilog Source Code

Below are skeleton code outlines for each module. Complete implementation should be substituted here.

gates_lib.v

```
// Gate Library with Propagation Delays
// Replace <DELAY> with values from Table 1 for your student ID

primitive INV_gate(out, in);
```

```
output out; input in;
specify #(<DELAY>) (in ==> out) = 1; endspecify
table 0:1; 1:0; endtable
endprimitive

// ... NAND, NOR, AND, OR, XOR, XNOR defined similarly ...
```

full_adder_1bit.v

```
module full_adder_1bit(a, b, cin, sum, cout);
input a, b, cin;
output sum, cout;
wire w1, w2, w3;
XOR_gate g1(w1, a, b);
XOR_gate g2(sum, w1, cin);
AND_gate g3(w2, a, b);
AND_gate g4(w3, w1, cin);
OR_gate g5(cout, w2, w3);
endmodule
```

dff.v

```
module DFF(clk, D, Q);
input clk, D;
output reg Q;
always @(posedge clk)
Q <= D;
endmodule
```

Full implementation of `rca_4bit.v`, `multiplier_rca.v`, `cla_4bit.v`, `multiplier_cla.v`, and their respective testbenches should be inserted here in the final submitted report. *[Replace this placeholder with your actual Verilog code]*